

Regularization

STA326

Department of Statistics and Data Science

Mar. 25, 2025

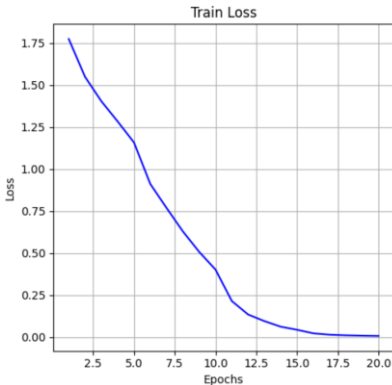
实验目标

1. 首先在model/mlp.py中实现一个4层的MLP，训练MLP并且观察过拟合现象。
2. 逐步添加各种正则化方法，包括Data Augmentation、Dropout、Batch Normalization (BatchNorm)、L2 Regularization、Early Stopping。
3. 通过实验分析不同正则化的训练效果

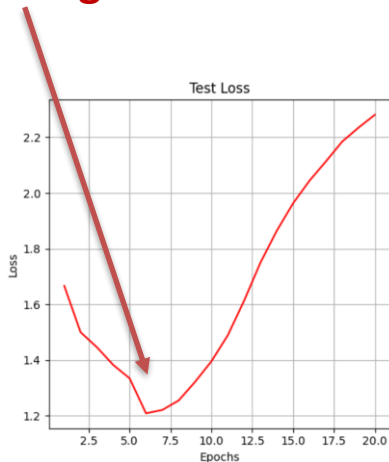


- 实验数据集：CIFAR-10
- 模型结构：简单的4层MLP网络模型
- 观察模型过拟合现象

```
class MLP(nn.Module):  
    def __init__(self):  
        super(MLP, self).__init__()  
        self.fc1 = nn.Linear(32 * 32 * 3, 512)  
        self.fc2 = nn.Linear(512, 1024)  
        self.fc3 = nn.Linear(1024, 512)  
        self.fc4 = nn.Linear(512, 10)  
  
    def forward(self, x):  
        x = x.view(-1, 32 * 32 * 3)  
        x = F.relu(self.fc1(x))  
        x = F.relu(self.fc2(x))  
        x = F.relu(self.fc3(x))  
        x = self.fc4(x)  
        return x
```



Overfitting!



在训练过程中，对输入数据进行变换，增加数据多样性，本质上是人为扩大训练集，增加数据的多样性。

- 水平翻转 `transforms.RandomHorizontalFlip(p=0.2)`
- 像素填充+随机裁剪 `transforms.RandomCrop(32, padding=4)`
- 颜色抖动，随机调整图像的亮度、对比度、饱和度和色调
`transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1)`
- 图像旋转 `transforms.RandomRotation(15)`

Data Augmentation

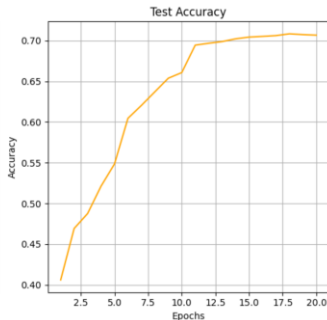
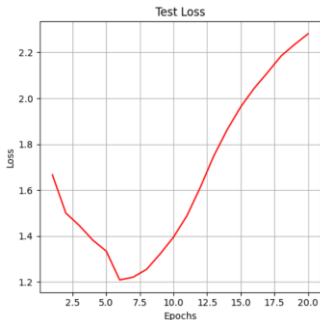
在训练过程中，对输入数据进行变换，增加数据多样性，本质上是人为扩大训练集，增加数据的多样性。

```
transform = transforms.Compose(  
    [  
        transforms.RandomHorizontalFlip(p=0.2), # 50% 概率水平翻转  
        transforms.RandomCrop(32, padding=4), # 先填充4个像素，再随机裁剪回 32x32  
        transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1)  
        transforms.RandomRotation(15), # 旋转 ±15 度  
        transforms.ToTensor(),  
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])
```

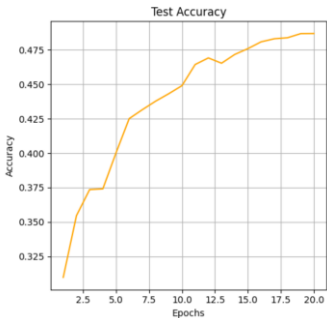
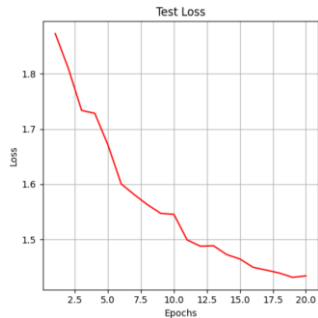


Data Augmentation

● 基准



● 增强后



思考：
Data Augmentation的
4种方式都有用吗？
参数值是最优的吗？

Dropout

在训练过程中，随机丢弃一定比例的神经元（即让它们的输出变为 0），防止神经元过度依赖特定特征：丢弃一部分神经元，让网络学会不同的路径，而不是记住训练数据。

```
class MLP(nn.Module):  
    def __init__(self, dropout_rate=0.5):  
        super(MLP, self).__init__()  
        self.fc1 = nn.Linear(32 * 32 * 3, 512)  
        self.fc2 = nn.Linear(512, 1024)  
        self.fc3 = nn.Linear(1024, 512)  
        self.fc4 = nn.Linear(512, 10)  
  
        # Dropout 层  
        self.dropout = nn.Dropout(dropout_rate)
```

```
def forward(self, x):  
    x = x.view(-1, 32 * 32 * 3)  
    x = F.relu(self.fc1(x))  
    x = self.dropout(x) # Dropout 防止过拟合  
    x = F.relu(self.fc2(x))  
    x = self.dropout(x)  
    x = F.relu(self.fc3(x))  
    x = self.dropout(x)  
    x = self.fc4(x) # 最后一层不加 Dropout  
    return x
```



Batch Normalization

在每个 batch 训练后，对数据进行归一化，BN 使得每一层的输入分布更稳定，加速训练并减少对特定 batch 的依赖。由于 BN 计算 mini-batch 的均值和方差，使得每个 batch 略有不同，提供一定的正则化效果。

```
class MLP(nn.Module):
    def __init__(self):
        super(MLP, self).__init__()
        self.fc1 = nn.Linear(32 * 32 * 3, 512)
        self.bn1 = nn.BatchNorm1d(512) # 添加 BN
        self.fc2 = nn.Linear(512, 1024)
        self.bn2 = nn.BatchNorm1d(1024) # 添加 BN
        self.fc3 = nn.Linear(1024, 512)
        self.bn3 = nn.BatchNorm1d(512) # 添加 BN
        self.fc4 = nn.Linear(512, 10)
```

```
def forward(self, x):
    x = x.view(-1, 32 * 32 * 3)
    x = F.relu(self.bn1(self.fc1(x)))
    x = F.relu(self.bn2(self.fc2(x)))
    x = F.relu(self.bn3(self.fc3(x)))
    x = self.fc4(x)
    return x
```



Early Stopping

在训练过程中，通过监控验证集的误差来决定何时停止。当验证集误差停止下降并开始上升时，立即结束训练。这能防止模型过度拟合训练数据的噪声，确保模型在未见数据上具有良好的泛化能力。

特性	Dropout	Early Stopping
手段	训练时随机“关闭”神经元	训练时实时监控误差曲线
目的	防止神经元间的过度依赖	防止训练迭代次数过多（过拟合）

总结

方法	对模型的影响	核心作用
Data Augmentation	增加数据多样性	从 源头 解决问题，让模型“见多识广”。
Dropout	随机切断神经元连接	强制模型学习 鲁棒 特征，防止神经元“结党营私”。
BatchNorm	标准化每一层的激活值	加速训练，同时具有微弱的正则化效果。
L2 Regularization	惩罚过大的权重参数	限制模型复杂度，让权重分布更 平滑 。
Early Stopping	适时终止训练过程	防止模型在后期开始“死记硬背”噪声。



总结

组合效应 $1 + 1 \neq 2$

1. 黄金搭档 (Dropout + BatchNorm)
2. 协同作战 (Data Augmentation + Early Stopping)

注意：同时叠加过强的 L2 惩罚和高比例的 Dropout，可能会导致模型欠拟合

结论：

没有万能的单一组合，只有最适合数据的配置。

1. **先跑通基准 (Baseline)**：观察过拟合现象，确认模型有足够的容量。
2. **逐步添加**：建议先加 BN 稳定训练，再加 Dropout 或 L2 缓解过拟合，最后用 Early Stopping 保底。
3. **动态平衡**：如果使用了强力的数据增强，可以适当降低 Dropout 的比例。